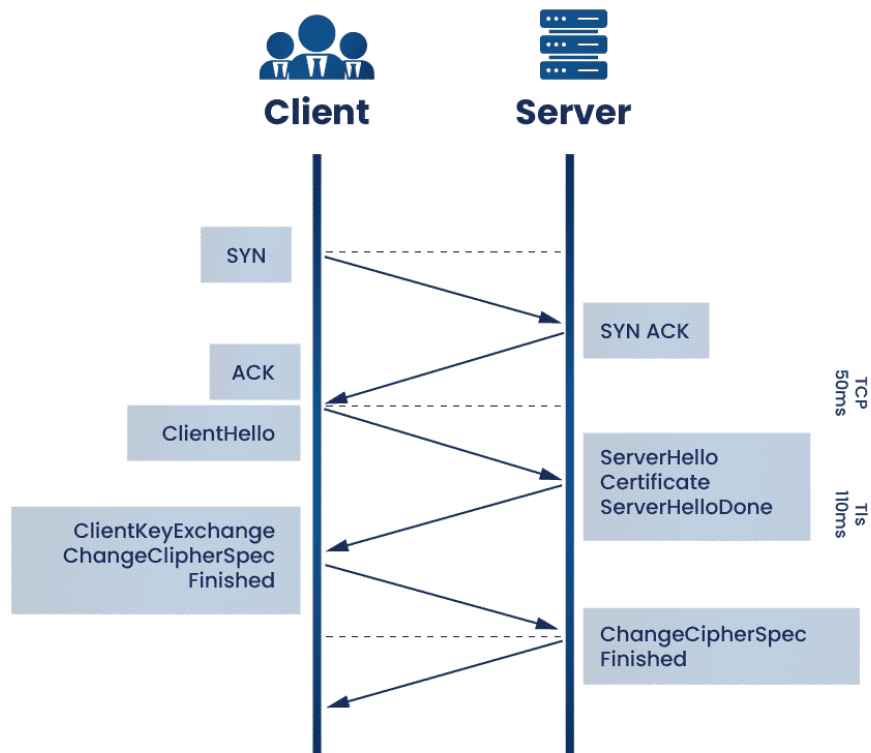


Connection & Security Fundamentals

Task 3.2 – TCP/IP Client-Server Model and TLS



*Dana Natsheh
Cyber Robot
IoT*

0. Executive Summary

This report covers the two questions this task set out to answer. First, in a long-lived TCP connection between an embedded client and a server, who opens the connection, who sits waiting for it, and why does the connection stay open for the lifetime of a session instead of being torn down and rebuilt for every single message – and how does that model differ from the request/response style of HTTP and the broker-mediated publish/subscribe style of MQTT (Section 1)? Second, once that TCP connection exists, what does wrapping it in TLS actually change about it – what new steps get added, what stays exactly the same – and why can a device not trust a TLS certificate at all until it has first synchronized its own clock over SNTP (Section 2)? Both sections close with a diagram, a comparison table, and a placeholder for a real-world capture the next hands-on task should fill in (e.g. a Wireshark trace or an ESP32 serial log).

1. TCP/IP Client-Server Model

1.1 Roles in a Long-Lived TCP Connection

The client is always the side that initiates. In the client-server model, a server is a program that starts, binds to a port, and then sits idle, listening for incoming connection attempts; it is always the client that reaches out first and opens the connection. The server does not need to know the client's address in advance, or even that the client exists, before that first connection attempt arrives – it is purely reactive. The client, by contrast, must know the server's address and port ahead of time, because it is the one doing the reaching-out.

“Listening” and “handling a connection” are two different jobs, kept on two different sockets. A server's listening socket is only ever used to accept new connection attempts; once a specific client's connection is accepted, the server hands that exchange off to its own dedicated connection socket, and the original listening socket immediately goes back to waiting for the next client, which is how one server can serve many clients without each new connection attempt blocking on whichever client connected first.

The connection stays open because reconnecting per message would throw away exactly the work that makes TCP worth using. Establishing a TCP connection costs a three-way handshake (SYN, SYN-ACK, ACK) before a single byte of application data can move, and for a device sending frequent, small updates – sensor readings, heartbeat pings, control commands – paying that handshake cost again for every message would multiply both the latency and the network overhead many times over. Keeping the socket open lets the device reuse the same connection, and the same negotiated TCP state (sequence numbers, congestion window, and – once added – the TLS session keys from Section 2) for as long as the session needs to run, which is also why TCP's own reliability mechanism matters here: every segment sent over that connection requires an acknowledgement, and TCP will automatically retransmit anything that goes unacknowledged rather than silently dropping it, which only works because the connection's state persists between messages instead of being reset each time.

1.2 How This Differs from HTTP Request/Response

Plain HTTP, in its original form, is a stateless, one-shot protocol layered on top of the very TCP connection described above. A classic HTTP exchange opens a TCP connection, sends one request, waits for one response, and closes – the server keeps no memory of the client between requests, so each request has to carry everything the server needs to understand it. Because HTTP is stateless, it cannot recover context after an abnormal disconnection, and getting continuous updates out of it requires the client to keep asking again (polling) rather than the server pushing data the moment it is available, which adds both latency and unnecessary network traffic compared to a connection that is simply left open. (Modern HTTP/1.1 keep-alive and HTTP/2 reuse a single TCP connection across several requests, narrowing this gap in practice, but the request/response *pattern* itself – client asks, server answers, nothing flows unless it was asked for – is unchanged.) The long-lived TCP session in Section 1.1 is the more general case HTTP is built on top of: a raw socket that either side can write to at any time, with no requirement that every message be paired with a matching request.

1.3 How This Differs from MQTT's Broker-Mediated Pub/Sub

MQTT keeps the long-lived TCP connection but removes the direct client-to-client link entirely. Every MQTT device – sensor, controller, dashboard – is a client, but no client ever addresses another client directly; each one holds its own long-lived TCP connection to a third party, the broker, and all communication passes through it. A publisher does not send a message to a specific recipient; it publishes to a named topic, and the broker is solely responsible for matching that topic against every current subscription and forwarding a copy of the message to each interested subscriber. This decouples the publisher and subscriber in three ways at once: neither needs to know the other's address or even that the other exists (space), neither needs to be connected at the exact same instant for a message to eventually arrive, since the broker can hold undelivered messages for a disconnected client depending on the quality-of-service level in use (time), and neither blocks waiting on the other, since publishing and receiving happen independently on each client's own connection (synchronization). This is the opposite tradeoff from HTTP: HTTP's request/response model requires the client to know exactly which server to ask and to wait for that specific server's answer, while MQTT's broker removes that direct coupling in exchange for depending on the broker itself being reachable.

1.4 Comparison Table

Table 1: How a raw long-lived TCP session, classic HTTP request/response, and MQTT's broker-mediated pub/sub differ on the properties that matter for an embedded client.

Property	Raw long-lived TCP	HTTP request/response	MQTT pub/sub
Who initiates?	The client always opens the connection; the server only listens and accepts	The client always sends the request; the server only answers what was asked	Every client (publisher or subscriber) opens its own connection to the broker; clients never open a connection to each other
Direct link between endpoints?	Yes – one socket, both sides read/write on it directly	Yes – but the “conversation” is one request paired with one response, not free-form	No – a broker sits between every publisher and every subscriber; they are never linked directly

continued on next page

Table 1 continued from previous page

Property	Raw long-lived TCP	HTTP request/response	MQTT pub/sub
State kept between messages?	Yes – the whole point of keeping the socket open	Classic HTTP: none (stateless); each request is self-contained	Yes – the broker keeps session state (subscriptions, undelivered messages) per client
Who can push data?	Either side, at any time	Only the server, and only in reply to a request – the client cannot be pushed to unprompted	Any publisher, at any time; the broker fans it out to subscribers without a subscriber asking first
Cost of frequent small updates	Low – handshake paid once, then messages flow on the open socket	High if reconnecting per request; lower with keep-alive, but still request-shaped	Low – one connection per client, topic-based fan-out, minimal per-message overhead by design

1.5 Connection Lifecycle Diagram

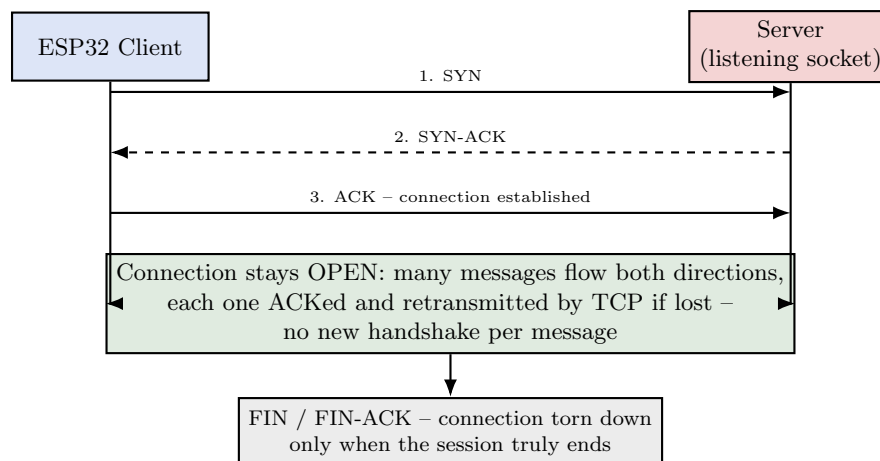


Figure 1: Lifecycle of a long-lived TCP connection: one handshake, an open session carrying many messages, and a single teardown at the end – contrasted with HTTP’s per-request open/close and MQTT’s broker-mediated fan-out described in Sections 1.2–1.3.

Interesting detail found during research: MQTT is often described as replacing the client-server model with pub/sub, but underneath, every single MQTT client-to-broker link is itself an ordinary long-lived TCP connection using the exact client/server roles from Section 1.1 – the client (publisher or subscriber) still initiates, the broker still listens and accepts. Pub/sub is a messaging pattern built *on top of* that TCP connection, not a replacement for it; what it removes is the requirement that the two application-level endpoints (publisher and subscriber) know about or connect to each other directly. This is also why an MQTT client can keep a keep-alive interval, prompting the broker to periodically check the connection is still alive – a concept that only makes sense because there is a persistent TCP connection underneath to check.

863	18:32:06.225084	146.66.71.198	10.0.0.31	TCP	66 80 → 9556 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
864	18:32:06.225212	10.0.0.31	146.66.71.198	TCP	54 9556 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0
865	18:32:06.225505	10.0.0.31	146.66.71.198	HTTP	388 GET / HTTP/1.1
871	18:32:06.272074	146.66.71.198	10.0.0.31	TCP	56 80 → 9556 [ACK] Seq=1 Ack=335 Win=30464 Len=0
873	18:32:06.279675	146.66.71.198	10.0.0.31	HTTP	739 HTTP/1.1 200 OK (text/html)
880	18:32:06.320004	10.0.0.31	146.66.71.198	TCP	54 9556 → 80 [ACK] Seq=335 Ack=686 Win=64768 Len=0
886	18:32:06.397447	10.0.0.31	146.66.71.198	HTTP	364 GET /images/netlab-labelled-pod1.jpg HTTP/1.1
889	18:32:06.449118	146.66.71.198	10.0.0.31	TCP	1514 80 → 9556 [ACK] Seq=686 Ack=645 Win=31488 Len=1460 [TCP s...
890	18:32:06.449119	146.66.71.198	10.0.0.31	TCP	1514 80 → 9556 [ACK] Seq=7146 Ack=645 Win=31488 Len=1460 [TCP s...

> Frame 865: 388 bytes on wire (3104 bits), 388 bytes captured (3104 bits) on interface 0
 > Ethernet II, Src: GemtekTe_39:f2:6f (ac:81:12:39:f2:6f), Dst: ArrisGro_93:6f:0c (04:4e:5a:93:6f:0c)
 > Internet Protocol Version 4, Src: 10.0.0.31, Dst: 146.66.71.198
 > Transmission Control Protocol, Src Port: 9556, Dst Port: 80, Seq: 1, Ack: 1, Len: 334
 > Hypertext Transfer Protocol

Figure 2: Wireshark capture of a client session on port 9556 to a server on port 80: packets 856, 863, and 864 are the SYN, SYN-ACK, and ACK of a single TCP handshake, and every message after that – the HTTP GET, the 200 OK, and the further GET requests – is exchanged on that same connection (same source/destination ports, continuing Seq/Ack numbers) rather than opening a new one each time.

2. TLS – What It Actually Adds

2.1 What Changes on Top of Plain TCP

TLS runs entirely inside the TCP connection that is already open – it does not replace the transport layer, it wraps it. The plain three-way TCP handshake from Section 1.5 happens first, exactly as before; only once that connection is established does the TLS handshake begin, negotiating its own parameters over the bytes TCP is already moving. Nothing about TCP itself – its port numbers, its acknowledgement and retransmission behaviour, its concept of an open socket – changes because TLS is present; TLS is a layer sitting between the application (HTTP, MQTT, or anything else) and the raw TCP stream.

The TLS handshake adds three things a plain TCP connection never had: negotiated parameters, a verified server identity, and a shared secret. A client opens with a ClientHello listing the TLS versions and cipher suites it supports, plus a random value; the server answers with a ServerHello choosing one of those options, plus its own random value and, critically, its certificate – a document containing the server’s public key, its domain name, and a signature from a certificate authority (CA) that the client is already configured to trust. The client checks that signature by building a chain from the server’s certificate up to a CA it already trusts; if that chain doesn’t resolve, or the certificate is outside its own validity window (Section 2.3), the handshake is aborted before any application data is sent. Once the certificate is accepted, both sides use the exchanged key material to independently derive the same symmetric session keys – asymmetric cryptography is used only to bootstrap trust and exchange this key material, while the actual application data afterward is encrypted with faster symmetric cryptography using the freshly derived keys.

What TLS does *not* change is the data format the application sends. Whatever bytes an HTTP request, an MQTT PUBLISH packet, or any other application message would have put on the wire in plain TCP are exactly the bytes that go into the encrypted TLS record – TLS adds a record-layer wrapper (encryption, and a MAC or AEAD tag for integrity) around that payload, but it does not reformat, reorder, or reinterpret the application protocol underneath it. This is the practical reason MQTT-over-TLS (*MQTTs*) and HTTP-over-TLS (*HTTPS*) exist as simple compositions rather than as separate protocols: the application layer is unchanged, only the channel it rides on gains confidentiality, integrity, and server authentication.

2.2 TLS Handshake Diagram

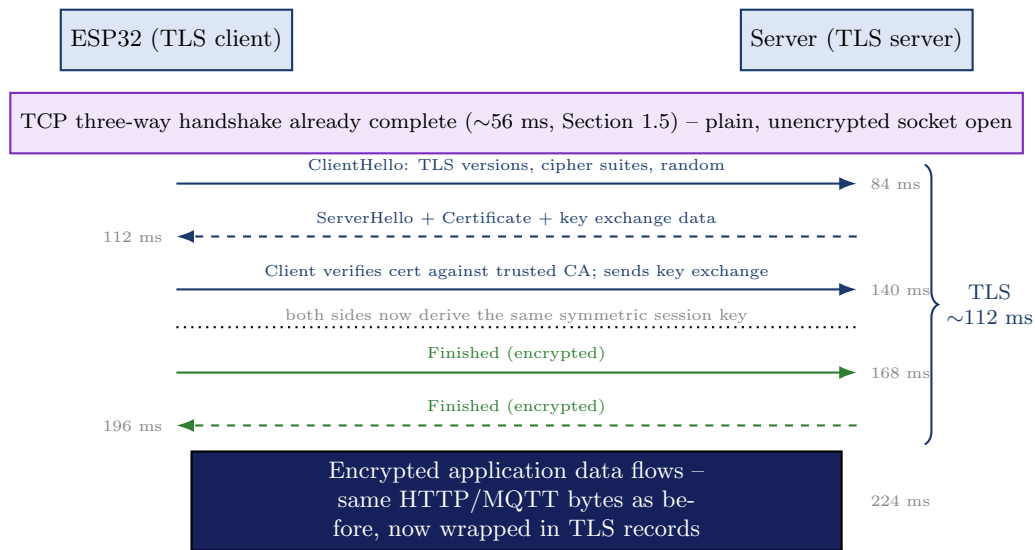


Figure 3: The TLS handshake happens after the TCP handshake and before any application data: it negotiates parameters, authenticates the server via its certificate, and derives a shared symmetric key – after which the application protocol continues completely unchanged, just encrypted. Elapsed times shown are typical round-trip figures for a full TLS 1.2 handshake, continuing on from the ~56 ms TCP handshake in Figure 1 – roughly another 110–170 ms before encrypted application data can flow.

2.3 Why Accurate Device Time Is a Prerequisite

A certificate is only valid inside a specific time window, so checking it requires knowing the current time. Every TLS certificate carries a *Not Before* and a *Not After* timestamp; during the handshake in Section 2.2, the client compares its own idea of the current time against that window, and rejects the certificate as not-yet-valid or expired if the comparison fails. This check happens automatically, as part of the handshake, and cannot be skipped by the application – which means a device with a wrong clock will reject a perfectly valid, unexpired certificate, or worse, accept one that has actually expired, purely because its own sense of “now” is wrong.

The ESP32 has no reliable idea of the current time on its own. Unlike a server that runs continuously and keeps a battery-backed real-time clock synced to a known-good time, a microcontroller like the ESP32 has a real-time clock that drifts and, after a power cycle or a cold boot, has no built-in knowledge of the current wall-clock time at all – it has to be told. SNTP (Simple Network Time Protocol) is the mechanism used to tell it: a lightweight client-server protocol, a simplified subset of full NTP, where the device sends a small request over UDP port 123 to a time server and learns back the current time along with timestamps that let it estimate network delay and correct for it. Because SNTP uses UDP rather than TCP and involves no certificates of its own, it is deliberately one of the cheapest possible network exchanges a freshly booted device can perform.

This creates a strict ordering, not just a good practice: a device must run SNTP before it can trust any TLS certificate. If certificate-validity checking is left enabled with an unset or wildly wrong clock, the very first HTTPS or MQTTS connection a freshly booted device tries to make can fail for a reason that has nothing to do with the server, the network, or the certificate itself – it fails because the device does not yet know what day it is. The practical sequence, then, is: boot, connect to Wi-Fi, send an SNTP request and set the internal RTC from the response, and only then begin the TLS handshake described in Section 2.2 – at which point the certificate’s validity window can actually be checked against a trustworthy clock.

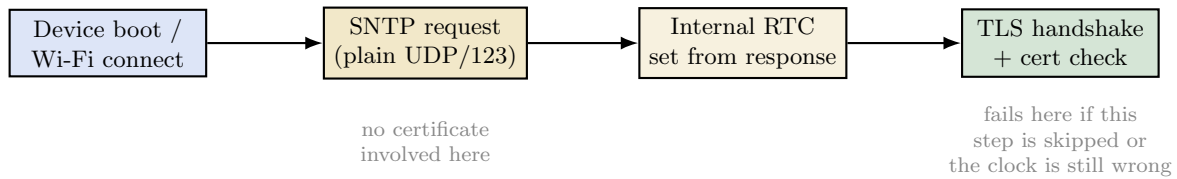


Figure 4: The required ordering: an unencrypted SNTp exchange sets the device’s clock before the TLS handshake attempts to check a certificate’s validity window against it.

```

I (8046) example_connect: Got IPv4 event: Interface "example_connect: sta" address: 192.168.10.26
I (8056) example_connect: Connected to example_connect: sta
I (8056) example_connect: - IPv4 address: 192.168.10.26
I (8066) example_connect: - IPv6 address: fe80:0000:0000:0000:86cc:a8ff:fe5e:5344, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (8076) example: Initializing SNTp
I (8076) example: Waiting for system time to be set... (1/10)
I (9316) example: Notification of a time synchronization event
I (10086) wifi:state: run -> init (0)
I (10086) wifi:pm stop, total sleep time: 4825292 us / 6492136 us

W (10086) wifi:<ba-del>idx
I (10086) wifi:new:<4,0>, old:<4,1>, ap:<255,255>, sta:<4,1>, prof:1
W (10096) wifi:hmac tx: ifx0 stop, discard
I (10166) wifi:flush txq
I (10166) wifi:stop sw txq
I (10166) wifi:lmac stop hw txq
I (10166) wifi:Deinit ll Desc rx mblock:10
I (10176) example: The current date/time in New York is: Mon Aug 29 04:50:12 2022
I (10176) example: The current date/time in Shanghai is: Mon Aug 29 16:50:12 2022
I (10176) example: Entering deep sleep for 10 seconds
ets Jun  8 2016 00:22:57
  
```

Figure 5: ESP32 serial-monitor log at boot: after connecting to Wi-Fi, the device initializes SNTp, waits for the time-synchronization event, and only then has a trustworthy clock (shown here printed for two time zones) – the exact prerequisite step Section 2.3 describes as needing to happen before a TLS handshake can validate a certificate’s Not Before/Not After window.

Interesting detail found during research: this dependency has a genuinely circular shape that is worth noticing rather than glossing over. Learning the correct time from the network normally ought to be done securely, but doing it securely would itself require TLS, which in turn requires knowing the time to validate a certificate – a chicken-and-egg problem some embedded-security writeups call the *time bootstrap problem*. The practical way out, and the reason SNTp is deliberately left unauthenticated and unencrypted in most ESP32 setups, is to accept an initial, untrusted plain-UDP time reading purely to get certificate-validity checking *into* a workable state, rather than trying to secure the very first time sync that makes all later security checks possible in the first place.

3. Summary & Checklist

Section 1 established that the client always initiates a TCP connection while the server only listens and accepts, and that the connection is kept open across many messages specifically to avoid paying the handshake cost repeatedly – a raw socket that either side can write to at will. Against that baseline, HTTP's classic request/response pattern is a stateless, client-initiated-only exchange layered on top of the same TCP connection, while MQTT keeps the long-lived TCP link but interposes a broker so that publishers and subscribers never connect to each other directly. Section 2 established that TLS adds a handshake of its own – parameter negotiation, certificate-based server authentication, and a derived shared key – entirely inside the TCP connection already open, without changing the application data format riding on top of it, and that this only works if the device's clock is already correct, which is why an unauthenticated SNTP exchange has to happen before the very first TLS handshake a freshly booted device attempts. The checklist for the next connected build is: keep the TCP connection open for the session rather than reconnecting per message; choose HTTP request/response for simple, occasional client-initiated queries and MQTT pub/sub for decoupled, frequent, many-to-many updates; wrap the connection in TLS wherever the data or the server's identity matters; and sync the clock over SNTP immediately after joining the network, before the first TLS handshake is attempted.

4. References

1. Dartmouth CS50, *Reading: Sockets – Client-Server Model*. <https://www.cs.dartmouth.edu/~tjp/cs50/reading/sockets/>
2. Engineering LibreTexts, *3.3: TCP Server*. [https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_Languages/Book:_Deep_into_Pharo_\(Bergel_Cassou_Ducasse_and_Laval\)/03:_Sockets/3.03:_TCP_Server](https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_Languages/Book:_Deep_into_Pharo_(Bergel_Cassou_Ducasse_and_Laval)/03:_Sockets/3.03:_TCP_Server)
3. Codefinity, *Learn Client Server Model – Networking Foundations in C*. <https://codefinity.com/courses/v2/a770d97f-7fa6-4e5e-8e5c-37734818ce0e/892f7722-ca34-4fe7-ba31-64c7fb7431d7/02c77c13-6352-48bb-a360-40bb139961c8>
4. EMQX, *Mastering MQTT: The Ultimate Beginner's Guide*. <https://www.emqx.com/en/blog/the-easiest-guide-to-getting-started-with-mqtt>
5. EMQ Technologies (Medium), *MQTT 5.0 – Introduction to the Publish-Subscribe Model*. <https://emqx.medium.com/mqtt-5-0-introduction-to-the-publish-subscribe-mode-dee7a0b5ff98>
6. HiveMQ, *MQTT Publish/Subscribe Architecture (Pub/Sub) – MQTT Essentials: Part 2*. <https://www.hivemq.com/blog/mqtt-essentials-part2-publish-subscribe/>
7. SCADA Protocols, *MQTT Publish/Subscribe Pattern Explained: How Pub/Sub Works*. <https://scadaprotocols.com/mqtt-publish-subscribe-pattern/>
8. purb03ragnarok (Medium), *Implementing MQTT Protocol on ESP32 for Publish/Subscribe Messaging*. <https://medium.com/@purb03ragnarok/implementing-mqtt-protocol-on-esp32-for-publish-subscribe-messaging-b5ac3e462806>
9. DEV Community (Labeeb Ahmad), *TLS Handshake Explained: How HTTPS Actually Works*. <https://dev.to/labeeb-ahmad/tls-handshake-explained-how-https-actually-works-1eo0>
10. Palo Alto Networks, *What Is the TLS Handshake? Process, Steps, and Best Practices*. <https://www.paloaltonetworks.com/cyberpedia/what-is-a-tls-handshake>

11. Checkly Docs, *TCP and SSL – What an Application Developer Needs to Know*. <https://www.checklyhq.com/docs/learn/monitoring/tcp-and-ssl/>
12. Gcore, *What Is an SSL Handshake? How TLS Protects Your HTTPS Connections*. <https://gcore.com/learning/ssl-handshake>
13. techinterview.org, *How the TLS Handshake Works, and What Interviewers Probe*. <https://www.techinterview.org/post/3233477262/tls-handshake-interview/>
14. Advanced Web Machinery, *ESP32 Time Bootstrap Problem*. <https://advancedweb.hu/esp32-time-bootstrap-problem/>
15. circuitlabs.net, *SNTP and Time Synchronization*. <https://circuitlabs.net/sntp-and-time-synchronization/>
16. Trustico, *How Time Synchronization Affects SSL Certificate Validation*. <https://shop.trustico.com/blogs/stories/how-time-synchronization-affects-ssl-certificate-validation-why-incorrect-clocks-cause-certificate-errors>
17. techtutorialsx, *ESP32: System Time and SNTP*. <https://techtutorialsx.com/2021/09/01/esp32-system-time-and-sntp/>
18. AWS, *FreeRTOS 202107.00 Includes SNTP Library for Time Information in IoT Applications*. <https://aws.amazon.com/about-aws/whats-new/2021/07/freertos-202107-00-includes-sntp-library-time-information-iot-applications>
19. ipSpace.net, *NTP Basics*. <https://blog.ipspace.net/kb/Internet/NTP/10-basics/>
20. TimeTools, *SNTP: All You Need to Know*. <https://timetoolsltd.com/ntp/sntp-overview/>
21. PentestPad, *Port 123 – NTP (Network Time Protocol)*. <https://www.pentestpad.com/port-exploit/port-123-ntp-network-time-protocol>